



Redacted

Miha Mitič in Jošt Eržen

Abstract

We present a domain-specific conversational assistant that helps users choose CPU (Central Processing Unit) and GPU (Graphics Processing Unit) components (and is extensible to other components) and answer related questions about PC hardware. The system combines a generative LLM (Large Language Models) with multiple Retrieval-Augmented Generation (RAG) indexes built from heterogeneous source material (component specifications and review articles). A lightweight embedding-based routing mechanism dispatches each query to one or more relevant indexes, and a query-rewriting step makes conversational follow-ups work with retrieval. We describe the design iterations that led to this architecture, the final implementation, and qualitative results.

Keywords

PC, hardware, computers, domain specific AI, assistant

Advisors: Aleš Žagar

Introduction

General-purpose LLMs[1] are convenient for casual hardware questions but tend to be unreliable when asked about specific components, current-generation performance, or comparative recommendations: their training cut-offs make them oblivious to recent releases, and they readily hallucinate specifications. We set out to build a small assistant that grounds its answers in retrieved, sourced material, while still feeling like a chatbot rather than a search engine.

Our final system we developed is a conversational PC-building assistant with three distinguishing properties. First, it uses *multiple* RAG[2] indexes built from qualitatively different source material rather than a single monolithic store. Second, queries are routed to the relevant index(es) through a cheap embedding-similarity step against per-index keyword descriptors, so unrelated material is never retrieved. Third, the routing layer is decoupled from the generation model and from the index contents, which lets new indexes be added without retraining or re-prompting anything.

Related Work

The components we use are standard: dense retrieval with sentence-level bi-encoders, FAISS[3] (Facebook Artificial Intelligence Similarity Search) for approximate nearest-neighbor search, and an instruction-tuned LLM for generation. The conversational rewriting step follows the well-known pattern of condensing chat history plus a new turn into a standalone

query before retrieval, which avoids the failure mode where pronouns or topic shifts derail the retriever. Routing across multiple retrieval sources (sometimes called *agentic* or *multi-index* RAG[4]) is also increasingly common; our contribution is not the idea but the specific, lightweight realization described below.

Dataset

The corpus is composed of two complementary source types, organised into separate CPU and GPU retrieval indexes.

Review articles. We assembled 49 hardware review articles - 29 covering CPUs and 20 covering GPUs - gathered from TechPowerUp[5], TechSpot[6], Tom's Hardware[7], Eurogamer[8], PC Gamer[9], PCMag[10], and Digital Trends [11]. Each article's HTML was downloaded through a browser, then converted to Markdown and split at H2-level section boundaries (e.g. *Gaming Performance*, *Thermals*, *Conclusion*), with fragments under 80 characters discarded as uninformative. Table 1 summarises the review corpus.

Structured data. Consumer CPU specifications for AMD and Intel processors were collected from the official AMD product specifications page[12] and Intel ARK[13]. GPU specifications were likewise sourced from AMD's product pages[12]. CPU and GPU benchmark scores were obtained from PassMark[14]. A list of popular Steam titles was retrieved from SteamDB[15]; minimum and recommended hardware requirements for each title were then fetched directly

from the Steam Store[16]. Each structured record is serialised as a single plain-text chunk and included in the respective index alongside the review chunks.

All chunks are embedded with BAAI/bge-base-en-v1.5 and stored in FAISS IndexFlatIP indexes; because embeddings are L2-normalised, inner product equals cosine similarity.

Quantity was the main limitation with written articles: specification data was not difficult to obtain in clean tabular form, but high-quality review articles are much harder to scrape at scale, and we ended up with fewer than we would have liked. This unevenness shows up in the results.

Table 1. Review article corpus statistics after chunking.

Category	Documents	Chunks
CPU	29	~290
GPU	20	~200
Total	49	~490

Exploration

The current architecture is the fourth iteration. In this section we describe the process that lead us to the final version. We list the failed ones because each one motivates a piece of the final design.

V1 – single index, specs only. The first prototype indexed only specification sheets into one FAISS store. It was good at factual spec questions (“how many cores does an X have?”) but useless for the questions users actually want answered, like “which of these two is faster for gaming?” – nothing in the spec sheet directly says that.

V2 – single index, specs and reviews mixed. Adding review articles to the same index fixed the recommendation problem but broke spec lookups: a question that asked for a single concrete number would often retrieve a long review paragraph that mentioned the part in passing, and the LLM would either answer vaguely or pick up an incidental claim from prose instead of a specification.

V3 – multiple indexes, LLM-based routing. The fix was to split the data back apart and let a router decide which index to query. Our first router asked the generation LLM (Mistral-7B-Instruct) to pick one or more index names from a list. This was both slow (an extra generation per turn) and unreliable: the model often picked the wrong index, refused to pick, or invented index names that did not exist. We assume using a bigger model both for generation and routing would help solve this issue, but this went against the goal of equipping a smaller otherwise worse model with information to make it compete with other larger and drastically slower models.

V4 – multiple indexes, embedding-based routing. The final design replaces the LLM router with a much smaller and faster sentence-transformer that scores the user query against a short natural-language description of each index. This is essentially free to run, deterministic, and trivially extensible:

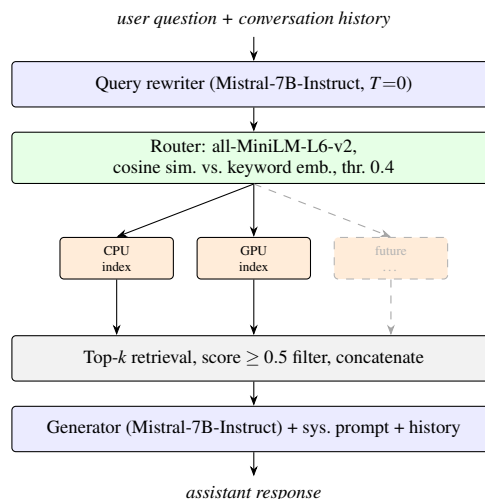


Figure 1. Single-turn pipeline. The user question and history are condensed into a standalone search query, routed to all indexes whose keyword-embedding cosine similarity passes the threshold (or the best single index if none do); score-filtered chunks from the selected indexes are concatenated into context, and the LLM produces a grounded response.

adding a new index just means writing a new description string or adding keywords.

Implementation

The pipeline (Figure 1) consists of four stages: query rewriting, query routing, retrieval, and generation. All stages share three frozen models: all-MiniLM-L6-v2 (384-dim) for query routing, BAAI/bge-base-en-v1.5 (768-dim) for dense retrieval and Mistral-7B-Instruct-v0.3 for rewriting and generation.

1. Query rewriting. The generation LLM is prompted with the conversation history and the new user message, and asked to emit a single standalone search query. Generation is deterministic (temperature=0, max 100 tokens). This step is what lets follow-ups like “and how does it compare to the other one?” actually retrieve useful chunks.

2. Routing. Each retriever carries a fixed embedding of a short keyword description (e.g. for the CPU index: “processor CPU cores threads clock speed compute tasks gaming workloads”). The user’s rewritten query is embedded with all-MiniLM-L6-v2 and cosine-compared against every retriever’s keyword embedding. All retrievers scoring above a threshold (we use 0.4) are selected; if none pass, we fall back to the single best-scoring index so the pipeline always has some context to work with. This both filters out irrelevant indexes and naturally allows multi-index retrieval when a question genuinely spans categories (e.g. “which CPU pairs well with an RTX 5070?”).

3. Retrieval. Each selected retriever runs a top- k search ($k=3$) over its FAISS index with the rewritten query, and dis-

cards results whose cosine similarity is below 0.5. Surviving chunks from all selected indexes are concatenated into the context block.

4. Generation. The final prompt to the LLM is a system message defining the assistant’s role and grounding rules, the prior conversation history (truncated to the last six turns), and a user turn that contains the retrieved context followed by the original question. The system prompt instructs the model to prefer the context, decline politely when context is irrelevant, and avoid speculating about specific parts.

The retrievers themselves are deliberately uniform: a RAGRetriever just bundles a FAISS index, its metadata, an embedding model, and a keyword vector. Adding, say, a motherboard or PSU index is purely a configuration change.

Results

Evaluation was qualitative. We compared the system against the same LLM using a similar prompt with no retrieval, on a small set of representative questions covering pure spec lookups, side-by-side comparisons, recommendation requests, and out-of-scope queries.

The routing layer behaved as intended. Single-category questions reliably activated only one index, comparison questions involving both a CPU and a GPU activated both, and clearly off-topic questions (“what’s the weather?”) landed below threshold and triggered the top-1 fallback, after which the min-similarity cutoff in the retriever returned no chunks and the assistant correctly declined to answer. Splitting the corpus into two indexes also clearly helped retrieval quality versus the V2 mixed index: spec questions returned spec chunks, recommendation questions returned review chunks.

The weaker points were data and model size. Review coverage was thin in several places, and answers there occasionally fell back to vague generalities because no good chunk was available. The 7B generation model also sometimes underused the context, restating the question rather than synthesising across multiple retrieved snippets. Both are addressable – more scraping for the former, a larger or better-instruction-tuned model for the latter – but were outside the scope of the seminar. Even so, against the no-RAG baseline the difference was substantial: the baseline confidently invented part numbers and incorrect specifications, while the RAG version stayed grounded and was willing to say it did not know.

Formal automatic evaluation (e.g. Recall@k on a held-out query set) is left for future work given the absence of a labelled QA benchmark for this narrow domain.

1. Conclusion

We built a small conversational PC-building assistant around a routed multi-index RAG architecture. The combination of cheap embedding-based routing, query rewriting for conversational use, and a clean separation between retrievers and the generation model worked well and is easy to extend. The

remaining quality ceiling is set mostly by the source data and by the size of the generation model rather than by the retrieval design itself.

References

- [1] Humza Naveed, Asad Ullah Khan, Shi Qiu, Muhammad Saqib, Saeed Anwar, Muhammad Usman, Naveed Akhtar, Nick Barnes, and Ajmal Mian. A comprehensive overview of large language models. *ACM Transactions on Intelligent Systems and Technology*, 16(5):1–72, 2025.
- [2] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474. Curran Associates, 2020.
- [3] Lucia D Krisnawati, Aditya W Mahastama, Su-Cheng Haw, Kok-Why Ng, and Palanichamy Naveen. Indonesian-english textual similarity detection using universal sentence encoder (use) and facebook ai similarity search (faiss). *CommIT (Communication and Information Technology) Journal*, 18(2):183–195, 2024.
- [4] Aditi Singh, Abul Ehtesham, Saket Kumar, Tala Talaie Khoei, and Athanasios V Vasilakos. Agentic retrieval-augmented generation: A survey on agentic rag. *arXiv preprint arXiv:2501.09136*, 2025.
- [5] TechPowerUp. TechPowerUp. <https://www.techpowerup.com/>. Accessed May 2026.
- [6] TechSpot. TechSpot – technology news, reviews and analysis. <https://www.techspot.com/>. Accessed May 2026.
- [7] Tom’s Hardware. Tom’s hardware – pc components, reviews, and news. <https://www.tomshardware.com/>. Accessed May 2026.
- [8] Eurogamer / Digital Foundry. Eurogamer. <https://www.eurogamer.net/>. Accessed May 2026.
- [9] PC Gamer. PC gamer. <https://www.pcgamer.com/>. Accessed May 2026.
- [10] PCMag. PCMag – the independent guide to technology. <https://www.pcmag.com/>. Accessed May 2026.
- [11] Digital Trends. Digital trends. <https://www.digitaltrends.com/>. Accessed May 2026.
- [12] Advanced Micro Devices. AMD processor and graphics specifications. <https://www.amd.com/en/products/specifications>, 2026. Accessed May 2026.
- [13] Intel Corporation. Intel® product specifications (ARK). <https://ark.intel.com>, 2026. Accessed May 2026.
- [14] PassMark Software. PassMark CPU and GPU benchmark charts. <https://www.passmark.com>, 2026. Accessed May 2026.

- [15] SteamDB. SteamDB — database of everything on Steam. <https://www.steamdb.info>, 2026. Accessed May 2026.
- [16] Valve Corporation. Steam store. <https://store.steampowered.com>, 2026. Accessed May 2026.